6 Fraqmentasiya məlumatın yayılmasıdır

İndeks məlumatı səhifələrdə saxlayır. Vaxt keçdikcə yeni sətirlər əlavə edilir, bəzisi böyüyür, bəzisi silinir. Səhifələrin sırası və doluluğu dəyişə bilər.

Fraqmentasiya barədə düşünəndə iki ayrı sualı ayır:

- Açar sırası ilə getdiyim səhifələr faylda hansı ardıcıllıqla yerləşir?
- Eyni məlumat neçə səhifəyə yayılıb və bu səhifələr nə qədər doludur?

Eyni məlumat üçün iki fərqli vəziyyət

```
Yaxsi doluluq:
    [dolu sehife] [dolu sehife]

Asagi doluluq:
    [yari dolu] [yari dolu] [yari dolu] [yari dolu]
```

İkinci vəziyyətdə oxunacaq və yaddaşda saxlanacaq səhifə sayı arta bilər. Bu, məlumatın yanlış yazıldığı və ya itdiyi demək deyil.

Fraqmentasiya performans mövzusudur. Corruption isə məlumat quruluşunun korlanmasıdır. Bunları qarışdırma.

Bu dərsin əsas hissəsi SQL Server rowstore indekslərinə aiddir. Onların səhifələri və B+ tree məntiqi indeks dərsində izah edildi.

Xarici və daxili fraqmentasiyanı ayıraq

Açar aralığına görə səhifələr A, B, C sırası ilə oxunmalıdır. Amma həmin səhifələrin fayldakı nömrələri belə ola bilər:

Məntiqi sıra

```
A: acarlar 1..20    -> faylda sehife 10
B: acarlar 21..40   -> faylda sehife 90
C: acarlar 41..60   -> faylda sehife 11

Oxuma yolu: 10 -> 90 -> 11
```

Açar sırası ilə fiziki səhifə ardıcıllığının uyğun gəlməməsi **logical fragmentation** adlanır. Çox vaxt buna xarici fraqmentasiya deyilir. Böyük aralıqları oxuyanda ardıcıl gətirmənin səmərəsinə təsir edə bilər.

Page density səhifənin doluluğudur. Səhifə daxilində çox boş sahə qalmasına daxili fraqmentasiya kimi baxılır. Eyni məlumat daha çox səhifə tutursa yaddaş və oxuma xərci arta bilər.

Əməliyyat sisteminin baza faylını disk blokları arasında necə yerləşdirməsi isə başqa səviyyədir. **File system fragmentation** buna aiddir. SQL indeksini yenidən qurmaqla ümumi disk defraqmentasiyası eyni əməliyyat deyil.

Yadda saxla: sıra problemi ilə boşluq problemini ayrı ölç. Eyni “fraqmentasiya” sözü onların təsirini eyniləşdirmir.

Page split necə yaranır

Bir səhifədə 10, 20, 30, 40 açarlı sətirlər saxlanır. Səhifədə yeni sətir üçün yer qalmayıb. İndi 25 açarlı sətir əlavə etmək istəyirik.

Baza əlavə səhifə ayırıb sətirlərin bir hissəsini ora köçürə bilər. Bu bölünməyə **page split** deyilir.

Bölünmədən əvvəl

```
Sehife A: [10, 20, 30, 40]  Yer yoxdur.
```

```
Bölünmədən sonra sadələşdirilmiş görünüş
Sehife A: [10, 20]
Sehife B: [25, 30, 40]
```

Yeni səhifə faylın başqa yerində yerləşə bilər. İki səhifədə əlavə boşluq qala bilər. Bölünmə əlavə yazma və log işi yaradır. Silinmələr və böyüyən sətirlər də səhifə doluluğunu dəyişə bilər.

Fill factor indeks qurularkən yarpaq səhifələrini hansı hədəf doluluqla yaratmağı bildirir. Məsələn, 90 gələcək yazmalar üçün təxminən boş yer saxlayır. Sonradan həmin səhifənin daim 90 faiz dolu qalması demək deyil.

Aşağı fill factor bəzi split-ləri azalda bilər. Amma başlanğıcdan daha çox səhifə yaradır. Buna görə bütün indekslərə kor şəkildə aşağı dəyər vermək düzgün deyil.

Nəyi ölçmədən qərar verməməliyik

SQL Server indeksin səhifə sayı, sıra uyğunsuzluğu və doluluğu barədə məlumat verə bilər:

T-SQL diaqnostika nümunəsi

```
SELECT
    page_count,
    avg_fragmentation_in_percent,
    avg_page_space_used_in_percent
FROM sys.dm_db_index_physical_stats
(
    DB_ID(), OBJECT_ID(N'dbo.Orders'),
    NULL, NULL, 'SAMPLED'
)
WHERE index_level = 0
  AND alloc_unit_type_desc = 'IN_ROW_DATA';
```

`page_count` səhifə sayıdır. `avg_fragmentation_in_percent` sıra uyğunsuzluğu göstəricisidir. `avg_page_space_used_in_percent` doluluq göstəricisidir. `index_level = 0` yarpaq səviyyəsini seçir.

`DB_ID()` cari bazanı, `OBJECT_ID` Orders obyektini müəyyən edir. `SAMPLED` nümunə götürməklə analiz aparır. `IN_ROW_DATA` sətir daxilində saxlanan məlumat hissəsidir. Sorğunun görünməsi üçün uyğun baza icazələri lazımdır.

Tək faizə baxma. Beş səhifəli indeksdə yüksək faiz praktik olaraq böyük problem olmaya bilər. Milyonlarla səhifəyə yayılmış boşluq isə daha əhəmiyyətli ola bilər.

Qərarın əsası: ölçü, doluluq və real sorğunun gördüyü iş birlikdə qiymətləndirilir. Diaqnostika özü də resurs sərf edir; hədəf cədvəldən başlamaq faydalıdır.

Reorganize və rebuild eyni iş üsulu deyil

Defraqmentasiya quruluşu daha əlverişli formaya gətirmək işidir. SQL Server-də iki əsas indeks baxımı üsulu var:

Üsul	Nə edir
REORGANIZE	Mövcud indeksin yarpaq səhifələrinin sırasını və sıxlığını mərhələli yaxşılaşdırır.
REBUILD	İndeksi yenidən qurur. Daha böyük resurs və əlavə sahə ehtiyacı yarada bilər.

T-SQL alternativ əmrlər

```
ALTER INDEX IX_Orders_Customer_Date
ON dbo.Orders REORGANIZE;

ALTER INDEX IX_Orders_Customer_Date
ON dbo.Orders REBUILD;
```

Bunlar bir-birinin ardınca mütləq işlədiləcək siyahı deyil, iki alternativ nümunədir.

Reorganize indeks statistikasını ayrıca yeniləmir. Rebuild müvafiq indeks statistikasını yeniləyə bilər, amma cədvəlin bütün ayrı sütun statistikalarını avtomatik düzəltmir. Statistika dəyərlərin paylanması barədə plan seçiminə kömək edən məlumatdır.

Online baxım işləyərkən istifadəçi girişini daha çox davam etdirməyə imkan verir. Bütün kilidlərin yox olması demək deyil. Dəstək versiya, indeks növü və məhsul paketindən asılıdır. **Offline** baxım daha çox bloklanma yarada bilər.

Nəticə: resurs xərci ilə gözlənən sorğu qazancını müqayisə etmədən bütün indeksləri rebuild etmə.

Hansı vəziyyətdə baxım fayda verə bilər

Sifariş hesabatı geniş tarix aralığını oxuyur və çoxlu səhifə gəzir. İndeksdə aşağı doluluq varsa daha sıx quruluş səhifə sayını azalda bilər. Burada baxımın real faydası ola bilər.

Başqa sorğu isə bir sifariş nömrəsini dərhal tapır. Yüksək fraqmentasiya faizi həmin dar axtarışın mütləq yavaş olduğunu göstərmir.

Köhnə şablonlarda 5 və 30 faiz kimi hədlər görünə bilər. Bunlar bütün sistemlər üçün dəyişməz qanun deyil. **Əvvəl yavaş sorğunu ölç, sonra problemin fiziki quruluşdan gəlib gəlmədiyini yoxla.**

Saxlama qurğusu da təsir edir. SSD elektron yaddaş əsasında, HDD isə mexaniki disk əsasında işləyir. Səhifələrin fiziki sırasının təsiri eyni olmaya bilər. Aşağı doluluğun daha çox səhifə və yaddaş xərci yaratması isə yenə qalır.

Columnstore sətirləri başqa qayda ilə, sütun əsaslı sıxılmış qruplarda saxlayan indeks ailəsidir. Orada silinmiş sətir payı və qrupların ölçüsü kimi başqa göstəricilər əhəmiyyətlidir. Rowstore fraqmentasiya faizini ora olduğu kimi tətbiq etmə.

Özünü yoxla

Rebuild-dən sonra faiz yaxşılaşdı, sorğu vaxtı dəyişmədi. **Bu mümkündür.** Yavaşlığın əsas səbəbi başqa yerdə ola bilər. Məqsəd tək faizi deyil, real işi yaxşılaşdırmaqdır.

PostgreSQL-də VACUUM və REINDEX fərqi

PostgreSQL bir sətir dəyişəndə köhnə versiyanı dərhal yox etməyə bilər. Çünki həmin versiyanı hələ oxuyan başqa iş ola bilər. Bir neçə sətir versiyası ilə paralel işi idarə etməyə **MVCC** deyilir.

Sadələşdirilmiş dəyişiklik

```
Evvel: Order 42, Total = 30.
Sonra: Order 42, Total = 35.

Kohne oxucu ucun 30 versiyasi hele lazim ola biler.
Yeni oxucu yeni qaydalara uygun 35-i gore biler.
```

Artıq heç bir aktiv işə lazım olmayan köhnə sətir versiyasına **dead tuple** deyilir. Tuple burada sətir versiyasıdır. Bu və başqa boşluqlar faydalı məlumatla müqayisədə saxlanmanı böyüdə bilər; artıq şişməyə **bloat** deyilir.

VACUUM təmizlənə bilən köhnə versiyaların yerini gələcək istifadə üçün azad edir. Adi VACUUM ümumən bütün faylı sıxıb boşluğu diskə qaytarmır. **ANALYZE** plan üçün statistikanı yeniləyir. **Autovacuum** bu işləri avtomatik aparan mexanizmdir.

VACUUM FULL cədvəli yenidən yazıb sıxlaşdıra bilər, amma güclü kilid və əlavə sahə tələb edir. **REINDEX** indeksin yeni quruluşunu yaradır. Bunlar bir-birinin adı dəyişmiş eyni əməliyyat deyil.

Yadda saxla: PostgreSQL versiya təmizliyi ilə SQL Server rowstore sıra fraqmentasiyasını eyniləşdirmə.